TennisVision: Player, Ball and Court Analysis in Broadcast Tennis Videos with YOLO26

Francesco Zompanti, Rosario Spaziante, Luca Panetta
Bachelor's Degree Programme in Applied Computer Science and Artificial Intelligence (ACSAI)
Course: AI-LAB – Computer Vision, Signal Processing, and Natural Language Processing
Department of Computer Science, Sapienza University of Rome
Matricola: 2012601, 2136455, 2136770
zompanti.2012601@studenti.uniroma1.it
spaziante.2136455@studenti.uniroma1.it
panetta.2136770@studenti.uniroma1.it

Abstract

This report presents TennisVision, a computer vision pipeline that analyzes broadcast tennis videos and automatically produces match statistics: shot count, shot speed, and player movement speed, rendered on an annotated video with a top-down minimap. The system combines three YOLO26-based components: a pose model trained to localize 14 court keypoints, a pretrained detector with built-in tracking for the two players, and a detector fine-tuned on a dedicated dataset for the tennis ball. Court keypoints are used to estimate a homography that maps image pixels to metric court coordinates, so that all speeds and distances are computed directly in meters without ad-hoc conversion factors. The raw ball track is refined with a piecewise parabolic smoother that removes outliers and bridges missed detections, and shots are detected as reversals of the ball motion along the court length. The fine-tuned ball detector reaches 0.90 mAP@50 on a held-out test set, and the court keypoint model is evaluated on more than two thousand unseen frames. The main contribution is the design and evaluation of the complete metric pipeline, together with the dataset preparation and fine-tuning of the two custom models.

Keywords: computer vision; object detection; pose estimation; YOLO; homography; sports video analysis.

1 Introduction

Quantitative match statistics such as shot speed, rally length, and player movement are a central tool in modern tennis coaching and broadcasting. Professional systems (e.g. Hawk-Eye) rely on multiple calibrated high-speed cameras and are not accessible to amateur players or analysts. The problem addressed by this project is the extraction of such statistics from a *single* ordinary broadcast video, with no camera calibration and no manual annotation.

The problem is challenging for several reasons. The ball is a small, fast, motion-blurred object that frequently disappears for several frames; the camera view is perspective-distorted, so pixel displacements do not translate directly into physical speeds; and broadcast footage contains distractors (line judges, ball kids, on-screen graphics) that confuse generic detectors. Existing open implementations typically address only one of these sub-problems, such as ball tracking with TrackNet [1], or rely on classical line-detection heuristics for the court that are fragile under varying surfaces and lighting.

This project builds a complete pipeline on top of the modern YOLO26 family [2]. The personal contributions are: (i) the preparation of a 14-keypoint court dataset in Ultralytics pose format, including the remapping of the source annotation order to a canonical metric court model, and the training of a YOLO26-pose court detector; (ii) the fine-tuning and comparison of two YOLO26 variants (nano and small) on a dedicated tennis-ball dataset; (iii) a piecewise parabolic ball-trajectory smoother designed around the physics of ball flight, used instead of a generic recursive filter; (iv) shot- and bounce-detection rules operating in metric court space and image space respectively; and (v) the integration of all components into a single reproducible pipeline with metric analytics and visualization.

The rest of the report is organized as follows. Section 2 reviews related work. Section 3 describes the pipeline. Section 4 presents the datasets, the training protocol, the results, and a critical discussion. Section 5 concludes the report.

2 Related Work

Object detection. Single-stage detectors of the YOLO family [3] reformulate detection as a single regression pass over the image and remain the standard choice for real-time applications. This project uses YOLO26 [2], the most recent Ultralytics release, both as a pretrained person detector and as the backbone fine-tuned for ball detection; the underlying convolutional design descends from residual architectures [4]. Pretrained weights come from large-scale benchmarks such as COCO [5], which include the *person* and *sports ball* classes; however, as shown in Section 4, the generic *sports ball* class is insufficient for broadcast tennis footage, which motivates fine-tuning.

Ball tracking in sport video. TrackNet [1] introduced a heatmap-based deep network for tracking small, fast balls in sports broadcasts, demonstrating that learned detectors outperform color/shape heuristics. Compared to TrackNet, this project uses a general-purpose detector fine-tuned on ten-

nis data, and moves the temporal reasoning into an explicit physics-motivated smoothing step, which is simpler to train and easier to inspect.

Court registration. Mapping the broadcast view to a canonical court model is a planar homography estimation problem [6]. Classical approaches detect court lines with edge/Hough heuristics and match them to the model; they are sensitive to surface color and lighting. Recent work replaces line detection with learned keypoint regression. This project follows the learned approach: a YOLO26-pose model regresses 14 court keypoints directly, and the homography is estimated from the detected points with RANSAC.

Compared to the cited works and to existing tutorial repositories, the distinguishing element of this project is the fully *metric* formulation: every downstream quantity (shot speed, player speed, shot detection) is computed in real-world meters after homography projection, which removes perspective bias from all statistics.

3 Methodology

The complete pipeline is shown in Fig. 1. It processes the input video in stages: court keypoint detection, homography estimation, player tracking, ball detection, trajectory smoothing, projection to metric court space, shot detection, statistics computation, and rendering.

Court keypoints and homography. A YOLO26-pose model is trained to regress 14 keypoints corresponding to the intersections of the court lines (corners of the doubles and singles court, service boxes, and center marks). The canonical court model uses official ITF dimensions (doubles width $10.97\,\mathrm{m}$, length $23.77\,\mathrm{m}$), with the origin in the top-left corner and axes oriented as in image coordinates. The homography H between detected image keypoints and their metric model positions is estimated with RANSAC. Keypoints are detected on *every* frame, which doubles as a court/no-court classifier: broadcast footage cuts away to close-ups and replays, and frames with fewer than 8 confident keypoints are excluded from all later stages (4 are the algebraic minimum for a homography, but a stricter count makes the classifier robust). The broadcast camera pans and zooms even within a rally, so a single homography frozen over the whole clip would drift; instead the per-frame keypoints are first stabilized by a short centered temporal median (an 11-frame window) applied independently inside each contiguous court-visible segment – which suppresses detector jitter while still following the moving camera and never mixing keypoints across a cut – and a homography is then estimated *per frame* with RANSAC. Brief detection dropouts (up to five frames) are bridged so that a single missed frame does not split one rally into two segments. On out-of-domain surfaces the per-frame fit can optionally be refined by snapping the projected court-line skeleton onto the painted white lines with chamfer ICP, correcting a residual overlay drift without retraining (discussed in Section 4.4). H and H^{-1} give the bidirectional pixel–meter mapping used by all later stages.

Player tracking. The two players are detected with the pretrained YOLO26x person class and associated across frames by the tracker built into the Ultralytics framework. To keep the small, low-contrast far player from being lost when the 1920×1080 broadcast is downsampled, detection is run at an increased inference resolution ($1280\,\mathrm{px}$ on the longest side); when even that misses a player on one baseline half – common on clay and grass – a fallback detector is run on a cropped, enlarged image of that court half at a lower confidence threshold, recovering the missing player without reprocessing the whole frame. The two players are then identified by their court *position* rather than by a fixed pair of track IDs: in each frame every detection is projected through H, the candidates inside the playable area are split by court half, and one player is chosen per half. This is robust to ball kids, line judges and spectators, who stand outside the court, and to the tracker reassigning a fresh ID after an occlusion or a frame-edge exit – a detection is accepted when it either continues the player's previous track ID or is spatially continuous with their last position, and short bounded gaps are linearly interpolated. The court half assigns the labels (player 1 near the camera, player 2 on the far side), and because track IDs do not survive camera cuts the selection is repeated independently within each contiguous court-visible segment. The foot point of each bounding box is projected through H to obtain the player position in meters.

Ball detection and smoothing. The ball is detected by a YOLO26 model fine-tuned on a single-class tennis-ball dataset (Section 4.1). Raw detections are noisy: false positives (e.g. white shoes, scoreboard graphics, and second balls near the fence) and gaps of several frames are common. Instead of a recursive filter, a conservative piecewise parabolic smoother is used. Detections implausibly far from the projected court quadrilateral are rejected first in image space, retaining a generous margin for airborne balls above the far baseline; repeated detections that remain within a small image neighbourhood over several frames are then removed as static hotspots. The remaining track is split at physically impossible jumps, sharp direction changes (hits and bounces), and long detection gaps; nearly stationary segments are discarded as static false positives. A robust quadratic $x(t), y(t)$ is then fitted per segment with joint two-dimensional residual trimming and evaluated only inside the time span supported by that segment. Consequently, short bounded gaps are bridged, whereas long or inconsistent gaps remain undefined instead of being filled by an unsupported curve. Smoothed positions are then projected to court space.

Shot detection and analytics. Working in court coordinates makes shot detection simple and robust: a hit is a reversal of the ball velocity along the court length (y axis), i.e. a sign change of $\dot{y}$ after a short moving-average smoothing, subject to a minimum speed and a minimum temporal gap between consecutive hits. Velocity is interpolated only inside a continuous ball-track segment, never across a long detection gap or camera cut; short gaps between an incoming and an outgoing fitted arc are also considered as contact candidates, and the first contact after a long pause can initialize a serve without requiring an incoming branch. The projected position of a ball high above the court is displaced towards the far side, which can produce spurious mid-flight reversals; a candidate is therefore localized at the closest ball–player contact in the image – at the racquet the ball overlaps the player in pixels regardless of its height, whereas in court meters the height-induced offset can reach several meters. Nearby duplicate candidates are merged and the physically required

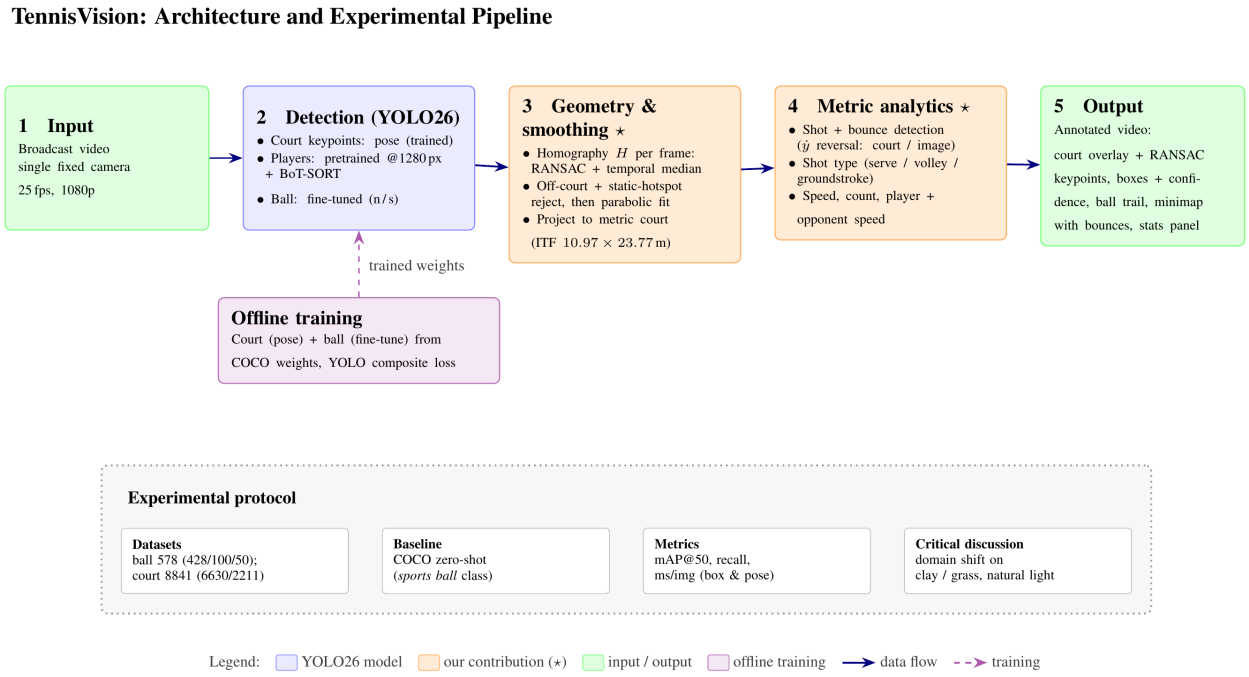

Figure 1: TennisVision pipeline. The input video is processed by three YOLO26 models: a pose model for the 14 court keypoints (from which the image-to-court homography is estimated), the pretrained detector with BoT-SORT association for the two players, and a fine-tuned detector for the ball. Ball detections are refined by a piecewise parabolic smoother, all positions are projected to metric court coordinates, shots are detected as motion reversals along the court length, and statistics are rendered on the annotated output video with a top-down minimap.

player alternation is enforced within each rally. The contact-stage player identity is preserved for the statistics instead of being recomputed from the height-biased court projection. Each shot is classified geometrically from the striker's metric position at the hit: a rally-opening hit struck from at or behind the baseline is a *serve*, a hit struck between the net and the service line is a *volley*, and everything else is a *groundstroke* – a classification that costs nothing because the positions are already in court coordinates. Ground bounces are detected separately, with a complementary signature in *image* space: gravity acts along the image vertical, so a bounce is a falling-to-rising reversal of the vertical pixel velocity that is not explained by a player strike (reversals close in time to a detected hit, or close in court space to either player, are rejected). The bounce instant is also the only moment when the ball lies on the court plane, hence the only moment when the homography returns its exact metric position – bounce locations are therefore the reliable anchor for in/out reasoning. Because positions are in meters, shot speed (km/h) and the per-frame player movement speed (computed over a 0.5 s window to suppress projection jitter) need no pixel-to-meter conversion factor. For each shot the system additionally records the opponent's movement speed at the moment of the hit, a simple proxy for how much the shot forces the opponent to move. Results are rendered as an annotated video: the court line skeleton reprojected from the canonical model through H (a direct visual check that the homography is correct) together with the detected court keypoints colour-coded by their RANSAC inlier/outlier status, which makes the robustness of the fit inspectable at a glance; player and ball boxes with their detection confidence; the fitted parabolic trajectory drawn as a fading trail behind the ball; a top-down minimap of the court showing the two players, the ball, and recent bounce locations; and a statistics panel reporting, per player, the live shot count, last shot speed and type, average shot speed, and current movement speed.

Reused vs. original components. The YOLO26 architecture, the pretrained weights, and the built-in tracker are taken from the Ultralytics framework [2] and used unmodified. The student's own contributions are the court-pose dataset conversion and keypoint remapping, the training of the court model and the fine-tuning of the ball models, the parabolic smoother, the metric shot/bounce-detection and analytics modules, and the overall pipeline integration.

4 Experimental Setup

Two models were trained: the ball detector (locally) and the court keypoint model (on Google Colab). Both are evaluated on held-out data.

4.1 Dataset

Ball dataset. A single-class tennis-ball detection dataset [7] of 578 broadcast frames with bounding-box annotations, split into 428 training, 100 validation, and 50 test images. Images come from broadcast footage, so the ball is typically only a few pixels wide and often motion-blurred, which is the main source of difficulty. The dataset was used with its original split.

Court keypoint dataset. The public TennisCourtDetector dataset [8], consisting of 1280×720 broadcast frames anno-

tated with 14 court keypoints each, provided as JSON: 6630 training and 2211 validation images. It was converted by the student to the Ultralytics pose format: each frame becomes one *court* instance whose bounding box is the convex extent of the visible keypoints, with normalized (x, y, vis) triplets per keypoint; frames with fewer than 4 visible keypoints are discarded, and the source keypoint order is remapped to the canonical metric court model. The 2211 validation images were never used during training and serve as the held-out evaluation set.

Table 1: Dataset summary.

Property	Value
Ball dataset (1 class)	578 images
train / val / test	428 / 100 / 50
Court dataset (14 kpts)	8841 images
train / held-out val	6630 / 2211
Input format	RGB frames

4.2 Training and Evaluation Protocol

The ball models were fine-tuned locally on an NVIDIA GTX 1650 (4 GB), which constrains the batch size to 4; the court-pose model was trained on Google Colab on an NVIDIA T4 (16 GB, batch 16). All training uses the Ultralytics framework on top of PyTorch, starting from COCO-pretrained YOLO26 weights, with the standard YOLO composite loss (box regression, classification, and distribution focal loss; plus keypoint losses for the pose model). Two ball variants were trained under identical settings to compare capacity against inference cost: YOLO26n (nano) and YOLO26s (small). Evaluation uses the standard COCO-style metrics: mAP@50 and mAP@50–95 (box for detection, keypoint OKS-based for pose), plus precision, recall, and per-image inference time measured on the same GPU.

Table 2: Training configuration (ball detector).

Parameter	Value
Framework	Ultralytics / PyTorch
Optimizer	auto (SGD)
Learning rate	0.01
Batch size	4
Epochs	100
Image size	640
Loss function	YOLO composite
Main metric	mAP@50

4.3 Results

Figure 3 shows the validation metrics of the two ball detectors during fine-tuning. Both converge smoothly within the 100 epochs, with mAP@50 saturating around epoch 60–70 while mAP@50–95 keeps improving slowly, consistent with the localization-bound regime discussed below. Table 3 compares the two fine-tuned ball detectors against their zero-shot baselines on the test split.

The zero-shot baseline is the same COCO-pretrained checkpoint evaluated before any fine-tuning, restricted to the *sports ball* class: it reaches at most 0.200 mAP@50 and 0.320 recall (YOLO26s), which quantifies how poorly the generic class transfers to broadcast tennis footage, where the ball is far smaller and blurrier than typical COCO instances. Fine-tuning on 428 images lifts mAP@50 by $+0.70$ and recall by $+0.56$ for the same architecture. Both fine-tuned variants reach high detection quality; YOLO26s outperforms YOLO26n on every quality metric on the test set (mAP@50 0.905 vs. 0.860, recall 0.880 vs. 0.805) at the cost of roughly 45% slower inference (17.8 vs. 12.3 ms/image). The moderate mAP@50–95 values (≈ 0.44–0.49) are expected for an object only a few pixels wide: at tight IoU thresholds even a one-pixel localization error is penalized, while for the downstream pipeline (trajectory smoothing in continuous coordinates) mAP@50-level localization is sufficient.

Table 3: Ball detection on the test split: COCO zero-shot baselines (*sports ball* class) vs. fine-tuned models. Inference time depends only on the architecture, so the zero-shot rows $(-)$ share the timing of their fine-tuned counterpart of the same size.

Method	mAP@50	Recall	Time
YOLO26n zero-shot	0.064	0.160	–
YOLO26s zero-shot	0.200	0.320	–
YOLO26n fine-tuned	0.860	0.805	12.3 ms
YOLO26s fine-tuned	0.905	0.880	17.8 ms

The court keypoint model was evaluated on the 2211 held-out frames, reaching 0.994 pose mAP@50 and 0.994 pose mAP@50–95 (box mAP@50 0.994, pose precision 0.997, recall 0.995, 5.2 ms/image). The training curves in Fig. 2 (66 epochs) show that the pose mAP saturates near 0.99 within the first ten epochs and the validation loss tracks the training loss closely throughout, with no sign of overfitting – confirming that court keypoint regression on a static broadcast view is an essentially solved sub-problem given enough training data.

Qualitatively, the estimated homography projects the players and the smoothed ball track onto the minimap consistently across rallies, and the detected shots coincide with the visible hits in the annotated output videos. The rendered confidence labels also make the pipeline inspectable at a glance: frames where the ball marker carries no confidence value are exactly those bridged by the parabolic fit rather than detected, so the share of interpolated frames is directly visible in the output.

4.4 Critical Discussion

The comparison between the two ball variants shows a genuine capacity effect: YOLO26s improves recall by 7.5 points on the test set, which matters directly for the pipeline, since every missed detection widens the gaps the parabolic smoother must bridge. YOLO26n trades this for faster inference; given that the pipeline runs offline, YOLO26s is the better default, while YOLO26n remains a valid choice for near-real-time use.

The fine-tuning itself is clearly beneficial, and Table 3 quantifies it: the pretrained COCO *sports ball* class recovers less than a third of the balls (recall 0.320 at best), so no amount of downstream smoothing could compensate for the missing detections. The hardest conditions for the ball detec-

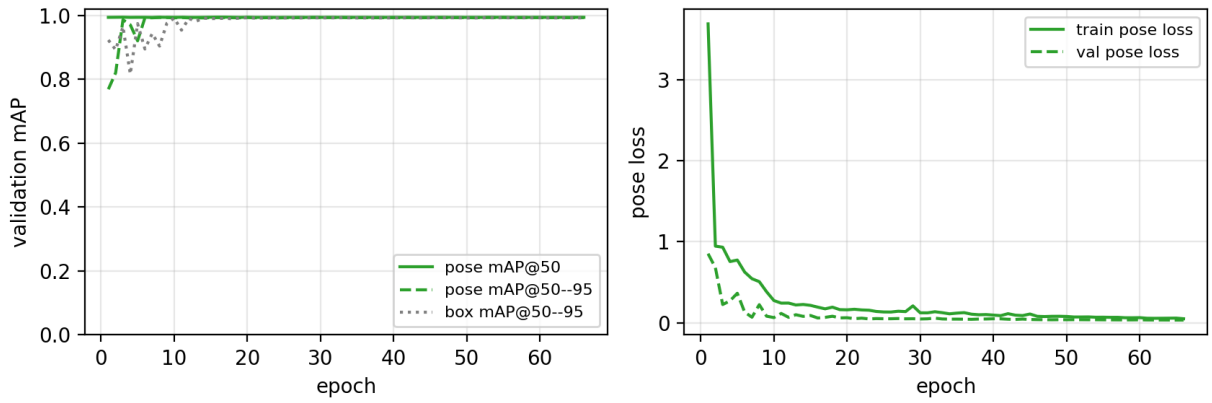

Figure 2: Training of the court keypoint model (YOLO26-pose, 66 epochs on a Colab T4 GPU). Left: validation mAP per epoch (pose mAP@50 solid, pose mAP@50–95 dashed, box mAP@50–95 dotted). Right: pose loss on the training (solid) and validation (dashed) splits.

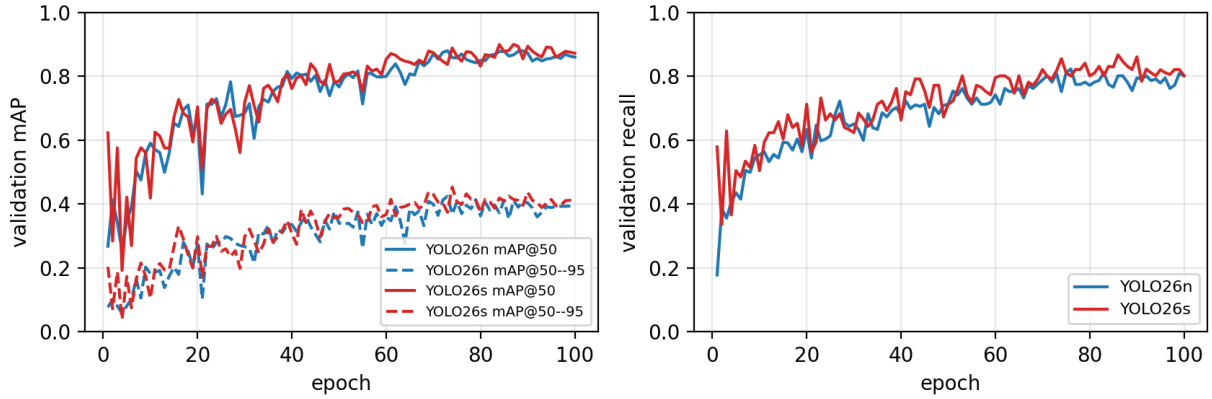

Figure 3: Fine-tuning of the two ball detectors on the validation split. Left: mAP@50 (solid) and mAP@50–95 (dashed) per epoch. Right: recall per epoch. YOLO26s (red) consistently edges out YOLO26n (blue) at equal training settings.

tor are fast serves and shots near the players' bodies, where motion blur stretches the ball into a faint streak; these are exactly the frames the smoother is designed to recover by interpolating within a parabolic segment. The main failure modes of the full pipeline are: rallies with very long detection gaps, where no parabolic segment can be fit and shot speeds become unavailable; and shots with little displacement along the court length (e.g. sharp cross-court angles), which weaken the velocity-reversal signal used for hit detection. Camera cuts and replays are instead handled explicitly: the per-frame court classification masks out non-court frames, the parabolic fit is prevented from bridging cuts, and player selection is repeated per court-visible segment.

These observations are coherent with the related work: as in TrackNet [1], the bottleneck is small-object detection under motion blur rather than association or geometry. The main limitations of the project are the moderate size of the ball dataset, the single-camera setup (no ball height, so reported shot speeds are projections on the court plane), and the heuristic nature of the event rules, whose thresholds are tuned on broadcast footage.

Domain shift across surfaces and lighting. The most significant limitation we observed is poor generalization beyond the conditions that dominate the training data. The court-keypoint and player components were exercised mainly on hardcourt footage under good (typically artificial) lighting, and they degrade markedly on clay and grass under natural light, with surface-specific failure modes; the fine-tuned ball detector is the least affected, since its training set already spans hardcourt, clay and grass frames. This must qualify the near-perfect court-keypoint scores of Section 4.3: the 0.994 pose mAP is an *in-distribution* result on held-out frames drawn from the same broadcast style, not evidence of cross-surface robustness; on clay and grass the keypoint regression becomes unreliable, and the estimated homography with it. The player detector suffers the same shift, and which player is lost depends on the surface: on red clay the far player – small and low-contrast against a saturated background – is frequently missed, whereas on grass only the far (top, P2) player is tracked consistently while the near player (P1) drops out; raising the inference resolution, and adding a fallback detector on an enlarged crop of the affected court half, recovered the missing player only partially, confirming that the gap is one of training distribution rather than input scale. These failures propagate to the analytics, because the event rules presuppose both a valid homography and two tracked players: on clay the shot count and the geometric shot-type classification (serve/volley/groundstroke) can no longer be produced at

all, and ground bounces – detected in image space but filtered against the court projection – are likewise no longer reliably identified once the homography degrades off-hardcourt. We also attempted several lightweight, training-free mitigations: CLAHE contrast normalization of the input frames; lowering the keypoint-confidence threshold so that more candidate points reach RANSAC; and a chamfer-ICP step that snaps the projected court lines onto the detected white lines (isolated by a surface-agnostic top-hat filter rather than a brightness threshold). None restored reliable cross-surface operation: the lower threshold in fact *hurt*, since the extra spurious keypoints were not all rejected and biased the homography; and the line refinement only removes a residual overlay drift once the keypoint fit is already approximately right, so it cannot rescue the grossly wrong poses produced off-hardcourt. The common root cause is the hardcourt bias of the court-keypoint training data, and the only effective remedy is a more surface-diverse court dataset – the direction taken up in the future work below.

Future work. Each limitation suggests a concrete next step. Cross-surface robustness would most directly be addressed by a more surface- and lighting-diverse court-keypoint dataset, since it is the court-pose model – not the ball detector, whose data already spans surfaces – that breaks off-hardcourt; the ball detector would still benefit from more data to close the residual recall gap on fast, blurred balls. The planar-speed limitation of the single-camera setup could be lifted by recovering the ball height, either through a multi-camera setup or a model that estimates ball elevation from the parabolic image trajectory, turning the current court-plane projections into true 3D speeds. The event-detection rules, currently the least transparent part of the system – their thresholds are tuned manually and never surfaced in the annotated output – could be replaced by a learned classifier over the smoothed trajectory, removing the manual tuning and adapting automatically to different footage. Robustness to long detection gaps could be improved by integrating an explicit temporal tracker (e.g. a TrackNet-style heatmap network [1]) downstream of the detector, so that rallies with sparse detections still yield continuous trajectories. Finally, the metric reliability of bounce locations – the only frames where the homography returns the exact ball position – could be exploited for automatic in/out line-calling, extending the system from statistics to officiating support.

5 Conclusion

This project addressed the extraction of quantitative match statistics from single-camera broadcast tennis videos. A complete pipeline was built around three YOLO26 models: a custom-trained court keypoint pose model, a pretrained player tracker, and a ball detector fine-tuned on a dedicated dataset, with all analytics computed in metric court coordinates via homography. Experimentally, the fine-tuned YOLO26s ball detector reached 0.905 mAP@50 and 0.880 recall on a held-out test set, outperforming the nano variant at a modest inference cost, and the court model was validated on 2211 unseen frames. The student's contributions span dataset preparation, model training, a physics-motivated trajectory smoother, metric shot and bounce detection, and the integration of the full system. The main limitations are the poor generalization beyond hardcourt and good lighting – a dataset-bias effect that degrades court, player and bounce detection on clay and grass – the planar speed estimates inherent to the single-camera setup, and the sensitivity to long ball-detection gaps; future work includes training the court-keypoint model on a more surface-diverse dataset, recovering the ball height for true 3D speeds, replacing the heuristic event rules with a learned classifier, and exploiting the metric bounce locations for automatic in/out line-calling.

References

[1] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. İk, and W.-C. Peng, "TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications," in *IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 2019.

[2] G. Jocher and the Ultralytics team, "Ultralytics YOLO." `https://github.com/ultralytics/ultralytics`, 2023–2026.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[5] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, pp. 740–755, 2014.

[6] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd ed., 2004.

[7] V. Dhanwani, "Tennis ball detection dataset." `https://universe.roboflow.com/viren-dhanwani/tennis-ball-detection`, 2023. Roboflow Universe.

[8] Yastrebksv, "TennisCourtDetector: Deep learning network for detecting tennis court keypoints." `https://github.com/yastrebksv/TennisCourtDetector`, 2023.